

Module 2 Lab 3 — Textbook Problem 4.4

Temperature categories: teaching the computer to make judgment calls with `if / elif / else`

Name: Roey Kadur GitHub repo: [**https://github.com/roee0910/cs82a-portfolio/tree/main/module2**](https://github.com/roee0910/cs82a-portfolio/tree/main/module2)

Problem statement (summary)

Given a list of temperatures, label each one (e.g. freezing / cold / mild / warm / hot) using an `if / elif / else` chain inside a loop, and print every temperature with its label. Because the chain is checked top to bottom and stops at the **first** matching condition, the order of the conditions matters, and boundary values (a temperature that sits exactly on a cutoff) need to be tested explicitly to make sure the code actually does what the stated rule promises.

Cutoffs (stated before the code, in degrees Fahrenheit)

Condition	Label
<code>temp <= 32</code>	freezing
<code>32 < temp <= 50</code>	cold
<code>50 < temp <= 65</code>	mild
<code>65 < temp <= 85</code>	warm
<code>temp > 85</code>	hot

Rule for boundary values: each cutoff value belongs to the *lower* (cooler) category, because every condition in the chain below is written with `<=`. For example, 65°F is **mild**, not warm -- warm only starts strictly above 65°F.

```
In [1]: def label_temperature(temp):  
        if temp <= 32:  
            return "freezing"  
        elif temp <= 50:  
            return "cold"  
        elif temp <= 65:  
            return "mild"  
        elif temp <= 85:  
            return "warm"  
        else:
```

```

        return "hot"

temperatures = [10, 32, 33, 50, 51, 65, 66, 85, 86, 100]

print(f"{'temp (F)':>10}    label")
for t in temperatures:
    print(f"{'t':>10}    {label_temperature(t)}")

```

temp (F)	label
10	freezing
32	freezing
33	cold
50	cold
51	mild
65	mild
66	warm
85	warm
86	hot
100	hot

Defending the boundary cases

The list above deliberately includes every cutoff value itself (32, 50, 65, 85) right next to the value one degree above it (33, 51, 66, 86), so the table proves the cutoff lands where the stated rule says it should:

- **32°F -> freezing**, but 33°F -> cold (freezing stops exactly at 32)
- **50°F -> cold**, but 51°F -> mild
- **65°F -> mild**, but 66°F -> warm
- **85°F -> warm**, but 86°F -> hot

This matters because of *how* `if / elif / else` chains work: the chain is evaluated top to bottom, and it stops at the **first** condition that is `True`. Since the first condition is `temp <= 32` (not `temp < 32`), a temperature of exactly 32 satisfies it immediately and is never even checked against the later conditions -- which is exactly why 32 is "freezing" and not "cold". If the conditions had instead been written with strict `<` everywhere, 32 would fall through to the `cold` branch instead, since it would no longer match the first condition. Order and the choice of `<=` vs `<` are what fixes where each boundary value actually lands, not just the number itself.

```

In [2]: boundaries = [32, 50, 65, 85]
print("Boundary check -- each cutoff value itself falls into the LOWER category")
for b in boundaries:
    print(f"    {b} F ->", label_temperature(b))

```

```

Boundary check -- each cutoff value itself falls into the LOWER category:
32 F -> freezing
50 F -> cold
65 F -> mild
85 F -> warm

```

Reproducibility notes

- This notebook was run top to bottom with **Kernel > Restart & Run All** before saving, so every code cell above shows its real output.
- A PDF copy (`module2_lab3.pdf`) was exported immediately after that full run, so the PDF shows the same outputs seen here.
- Repository: ****<https://github.com/roee0910/cs82a-portfolio/tree/main/module2>****